Legende:

Info: (Beispiel)

So ein Block beinhaltet allgemeine Informationen über die Unterrichtinhalte.

Achtung: (Beispiel)

So ein Block warnt vor mangelhaft ausgearbeiteten oder leicht irreführenden Unterrichtsinhalten.

Tipp: (Beispiel)

So ein Block beinhaltet fachliche Tipps für Übung, Praxis und Beruf.

Warnung: (Beispiel)

So ein Block warnt vor Gegenbeispielen, unwiderruflichem Fehlerpotenzial oder gefährlichem Code.

Zusatz: (Beispiel)

So ein Block beinhaltet Theorie die nicht auf dem Pflichtplan steht oder Aufgaben die über die Kursanforderungen hinausgehen.

Wiederholung: (Beispiel)

So ein Block beinhaltet Pflichttheorie die bereits behandelt worden ist.

 Nach dem grünen U-Boot folgt oberflächliche Theorie.

 Nach dem gelben U-Boot folgt leicht verständliche Theorie.

 Nach dem orangen U-Boot folgt tief-gehende Theorie.

 Nach dem roten U-Boot folgt sehr tiefe Theorie.

Die angegebene thematische Komplexität ist relativ zum Skriptkontext.

Info:

Benutzen Sie einen Reader der Seamless-Scrolling und Bookmarks untersütztz

Info:

Dieses Symbol ist stets eine Verlinkung zu einem für den Kontext relevanten GitHub-Repository bzw. -Branch.

Disclaimer:

WIP:

Dieses Material ist lizensiert.

Lizenz: CC BY-NC-ND 4.0 (wenn nicht anders angegeben)

- **Attribution (BY)**
- **Non Commercial (NC)**
- **No Derivatives (ND)**
- **No Warranty**
- **No Liability**

Einzelne Beispiele oder seperat geteilte Dokumente könnten einer anderen Lizenz unterliegen.

Autor: © Korobov Viktor, 2026

Netzwerke:

- LinkedIn
- GitHub
- YouTube
- LeetCode
- Steam

Dieses Kursmaterial ist ein lebendiges Dokument und wird mit einem KI Assisstenten gewartet.

Letzter Stand: 08.10.26.

Gebt aufmerksam Acht auf das Gesagte!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen

Einstieg in das Softwareengineering

Softwareentwicklung vs. Softwareengineering

Unter dem Begriff Softwareentwicklung wird der praktischen Prozess zur Erstellung von Softwarelösungen verstanden. Er umfasst alle operativen Schritte wie Planung, Entwurf (Design), Codierung, Test und Bereitstellung. Dabei steht die konkrete Umsetzung im Vordergrund.

Im Gegensatz dazu beschreibt Software Engineering einen strukturierten, wissenschaftlich fundierten Ansatz zur Entwicklung von Software. Ziel ist es, durch standardisierte Methoden, Modellierungstechniken und dokumentierte Prozesse eine möglichst wartbare, skalierbare und qualitativ hochwertige Software zu erstellen

Diese begriffliche Trennung hilft beim Verständnis, warum neben der reinen Codierung auch strukturierte Dokumentations- und Planungsmethoden, wie z. B. **UML**, essenzielle Bestandteile moderner Softwareentwicklung sind.

Info:

Auf **UML** wird im Detail im Verlauf folgender Veranstaltungen eingegangen.

Strategische Entwicklungsansätze

Strategische Entwicklungsansätze beschreiben die grundlegende organisatorische Ausrichtung von Entwicklungsarbeit. Sie legen fest, in welchem strukturellen Rahmen entwickelt wird, jedoch nicht, wie konkret

vorgegangen wird.

Bei der projektbasierten Entwicklung steht ein zeitlich begrenztes Vorhaben mit klar definiertem Ziel im Mittelpunkt. Die produktbasierte Entwicklung hingegen ist auf die langfristige Verantwortung und kontinuierliche Weiterentwicklung eines Produkts ausgerichtet.

Strategische Entwicklungsansätze bilden somit den übergeordneten Rahmen für die Wahl geeigneter Vorgehensmethoden.

Projektbasierte Entwicklung

Projektbasierte Entwicklung bezeichnet einen strategischen Entwicklungsansatz, bei dem ein klar abgegrenztes Vorhaben mit definiertem Anfang und Ende im Mittelpunkt steht.

Ziel ist die Erstellung eines konkret festgelegten Ergebnisses innerhalb vorgegebener Zeit-, Budget- und Qualitätsrahmenbedingungen.

Nach Abschluss des Projekts wird das Team in der Regel aufgelöst oder neu zugewiesen. Der Fokus liegt somit auf der einmaligen Zielerreichung.

Produktbasierte Entwicklung

Produktbasierte Entwicklung bezeichnet einen strategischen Entwicklungsansatz, bei dem ein dauerhaft bestehendes Produkt im Mittelpunkt steht.
Die Verantwortung des Teams endet nicht mit einer einmaligen Auslieferung, sondern umfasst die kontinuierliche Weiterentwicklung, Wartung und Verbesserung des Produkts über einen längeren Zeitraum.

Ziel ist es, den langfristigen Kundennutzen sowie den wirtschaftlichen Wert des Produkts nachhaltig zu steigern. Die Organisation richtet sich dabei typischerweise stabil um das Produkt herum aus, anstatt sich nach Abschluss einzelner Vorhaben aufzulösen.

Softwareprodukt

Softwareprodukte sind Softwarelösungen, die im Rahmen einer produktbasierten Entwicklung entstehen und langfristig bereitgestellt sowie weiterentwickelt werden. Sie sind nicht auf ein einmaliges Projekt beschränkt, sondern stehen dauerhaft im Mittelpunkt der organisatorischen Verantwortung.

Projektbasierte vs. Produktbasierte Entwicklung

Sowohl die projektbasierte als auch die produktbasierte Entwicklung sind fundamentale Ansätze in der Softwareentwicklung, die jeweils für unterschiedliche Anforderungen und Geschäftsmodelle optimiert sind.

Keiner der beiden Ansätze ist grundsätzlich besser oder schlechter. Vielmehr hängt die Wahl von den Zielen, dem Marktumfeld und den spezifischen Anforderungen eines Unternehmens oder Kunden ab.

Tabelle: Gegenüberstellung Planbasierter & Produktbasierter Entwicklung

Aspekt	Planbasiert	Produktbasiert
Ziel	Umsetzung eines Einzelvorhabens	Entwicklung eines langfristigen Produkts
Anforderungen	kommen vom Auftrafgeber	basieren auf Markt-/Nutzeranalyse
Laufzeit	begrenzt	langfristig, kontinuierlich
Finanzierung	Auftraggeber	Monitarisierung durch Lizenzen, Abos, etc
Vorgehensmodell	meist planbasiert oder hybride	agil

Projektmanagement und Produktmanagement im Vergleich

Projektmanagement

Projektmanagement fokussiert sich auf das "Wie". Es geht darum, ein klar abgegrenztes Vorhaben innerhalb eines festgelegten Zeit Ressourcen und Budgetrahmens erfolgreich umzusetzen.

Rolle eines Projektmanagers

Der Projektmanager ist verantwortlich für die Planung, Umsetzung und Kontrolle des Projekts. Er stellt sicher, dass das Projekt innerhalb der Zeit-, Kosten- und Qualitätsvorgaben bleibt.

Liste: Typische Aufgaben

- Erstellung eines Projektplans mit Meilensteinen
- Verwaltung von Ressourcen, Budgets und Risiken
- Koordination zwischen verschiedenen Teams und externen Partnern
- Einhaltung der Projektdeadlines und Budgetgrenzen

Produktmanagement

Produktmanagement ist strategisch ausgerichtet und betrachtet das "Was" und "Warum" eines Produkts. Der Produktmanager verantwortet die Vision, Marktpositionierung und Weiterentwicklung über den gesamten Lebenszyklus hinweg. Ziel ist es, das Produkt wirtschaftlich und nutzerzentriert weiterzuentwickeln.

Rolle eines Produktmanagers

Der Produktmanager ist verantwortlich für die Vision, Strategie und Entwicklung eines Produkts und stellt sicher, dass es die Bedürfnisse der Kunden erfüllt und für das Unternehmen profitabel ist.

Liste: Typische Aufgaben

- Entwicklung einer Produktstrategie und Roadmap
- Definition der Produktanforderungen basierend auf Kundenfeedback und Marktanalysen
- Zusammenarbeit mit Designern, Entwicklern und Marketingteams zur Umsetzung der Produktvision
- Optimierung des Produkts basierend auf Nutzerfeedback und **KPI**s
- Sicherstellung, des wirtschaftlichen Produkterfolgs

Operative Vorgehensmodelle in der Softwareentwicklung

Operative Vorgehensmodelle beschreiben die konkrete methodische Gestaltung und Steuerung des Entwicklungsprozesses. Sie legen fest, wie Anforderungen erfasst, geplant, umgesetzt und überprüft werden. Damit bestimmen sie die Abläufe, Rollen, Artefakte und Feedbackmechanismen innerhalb der Entwicklung.

Im Kontext operativer Vorgehensmodelle lassen sich insbesondere planbasierte und agile Ansätze unterscheiden.

Planbasierte Modelle setzen auf eine umfassende Vorabplanung und eine sequenzielle Struktur mit klar definierten Phasen.

Produktbasierte Modelle hingegen verfolgen einen iterativen und inkrementellen Ansatz, bei dem kurze Entwicklungszyklen, kontinuierliches Feedback und flexible Anpassungen im Vordergrund stehen.

Planbasierte Entwicklung

Planbasierte Entwicklung ist ein strukturierter und vordefinierter Softwareentwicklungsansatz bei dem der gesamte Entwicklungsprozess detailliert geplant wird, bevor mit der eigentlichen Implementierung begonnen wird.

Es gibt nur wenig Spielraum für Änderungen im Prozessverlauf. Geeignet ist dieses Modell vor allem für sehr klar definierte, stabile Projekte mit hohem Dokumentationsbedarf.

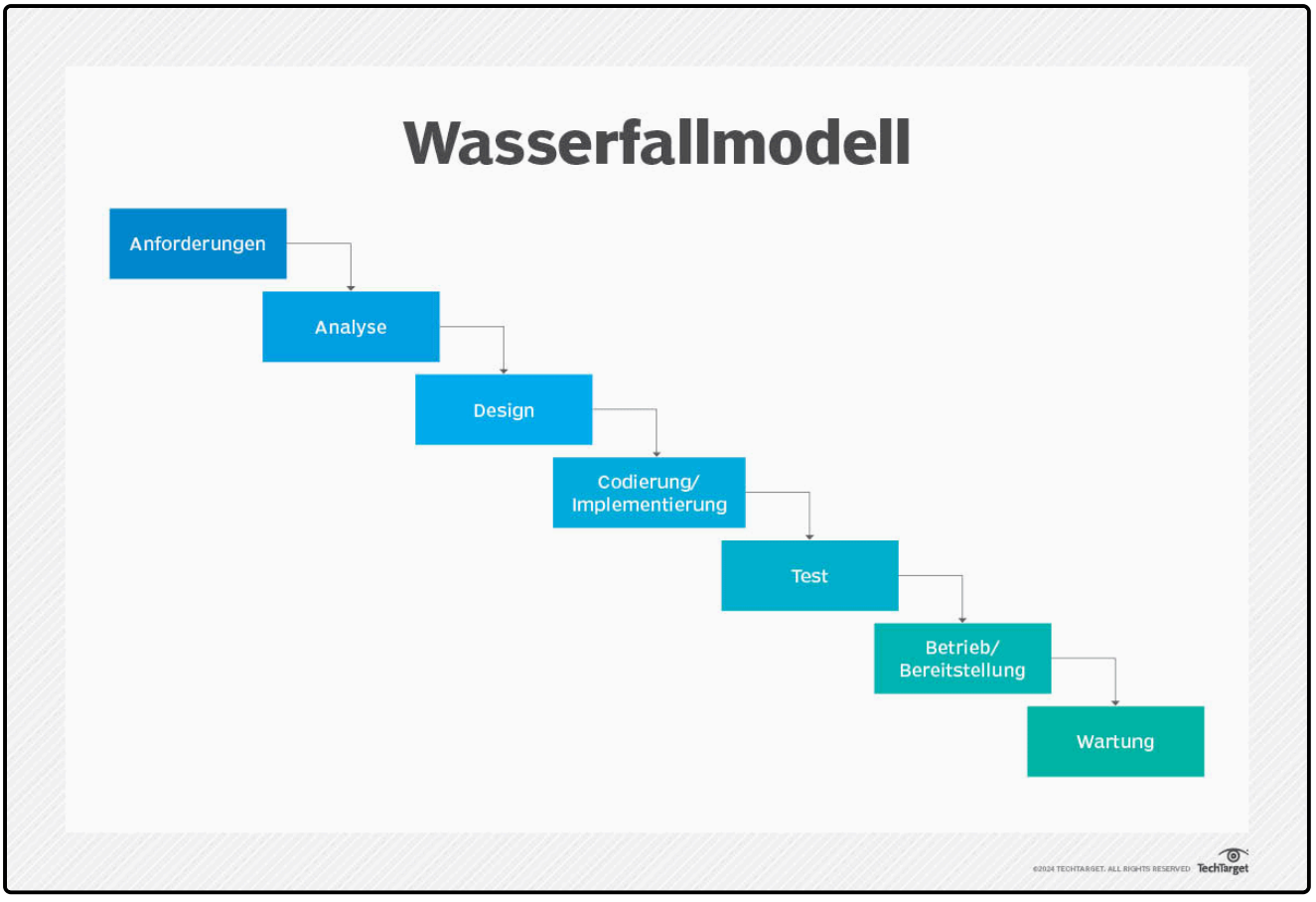
Dieses Modell wird oft in traditionellen, konkreten Vorgehensmethoden wie dem *Wasserfall*-Modell oder dem *V*-Modell verwendet.

Planbasierte Methoden sehen außerdem meist die Erstellung und Einhaltung von sogenannten Lasten- und Pflichtenheften vor. Diese bilden reachtlich verbindliche Vertragsbasis zwischen Entwickler und Aufftraggeber.

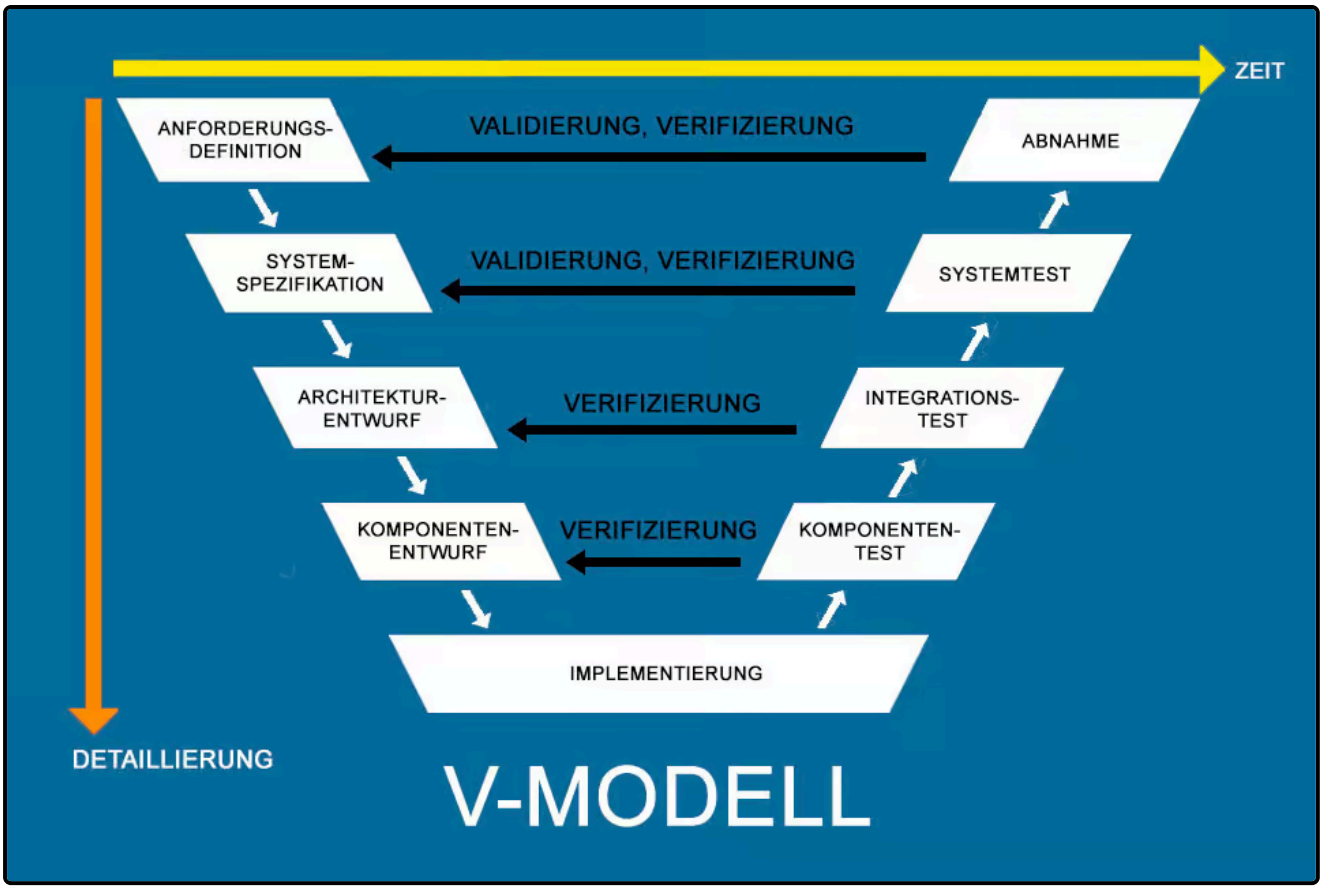
Info:

Konkrete planbasierte Entwicklungsmethoden (*Wasserfall*-, *V*-Modell, etc.) sind kein Bestandteil des Moduls und wird nicht näher konkretisiert.

Grafik: Wasserfall-Modell



Grafik: V-Modell



Lasten- und Pflichtenheft

Die Lasten- und Pflichtenhefte sind zentrale Dokumente zum Festhalten der Kunderanforderungen und des ausgemachten Projektumfangs in der planbasierten Entwicklung

Das Lastenheft beschreibt die Anforderungen an die Software aus Sicht des Auftraggebers Es enthält alle funktionalen und nicht-funktionalen Anforderungen, die das System erfüllen muss, ohne dabei die technische Umsetzung vorzugeben.

Das Pflichtenheft wird von den Entwicklern erstellt und beschreibt die konkrete technische Umsetzung der Anforderungen aus dem Lastenheft. Es enthält detaillierte technische Spezifikationen, Architekturentscheidungen und Umsetzungsmethoden.

Agile Entwicklung

Ein agiles Vorgehensmodell ist ein operatives Entwicklungsmodell, das auf iterativer, inkrementeller und anpassungsfähiger Arbeitsweise basiert. Die Entwicklung erfolgt in kurzen, wiederkehrenden Zyklen, in denen funktionsfähige Teilergebnisse erstellt, überprüft und bei Bedarf angepasst werden.

Im Mittelpunkt stehen kontinuierliches Feedback, enge Zusammenarbeit mit Stakeholdern sowie die flexible Reaktion auf sich ändernde Anforderungen. Ziel eines agilen Vorgehensmodells ist es, frühzeitig nutzbaren Mehrwert zu liefern und Unsicherheiten durch regelmäßige Überprüfung und Anpassung systematisch zu reduzieren.

Agile Enwicklung setzt auf Methoden wie *Scrum* oder *Extreme Programming* (**XP**) und somit auf inkrementelle, kontinuierliche Entwicklung in kurzen Zyklen Anforderungen und Lösungen entstehen durch enge Zusammenarbeit im Team.

Ein wichtiger Bestandteil agiler Entwicklung ist der Einsatz von **CI/CD** (Continuous Integration/Delivery)-Ansätzen, um Codeänderungen automatisch zu testen und direkt bereitzustellen.

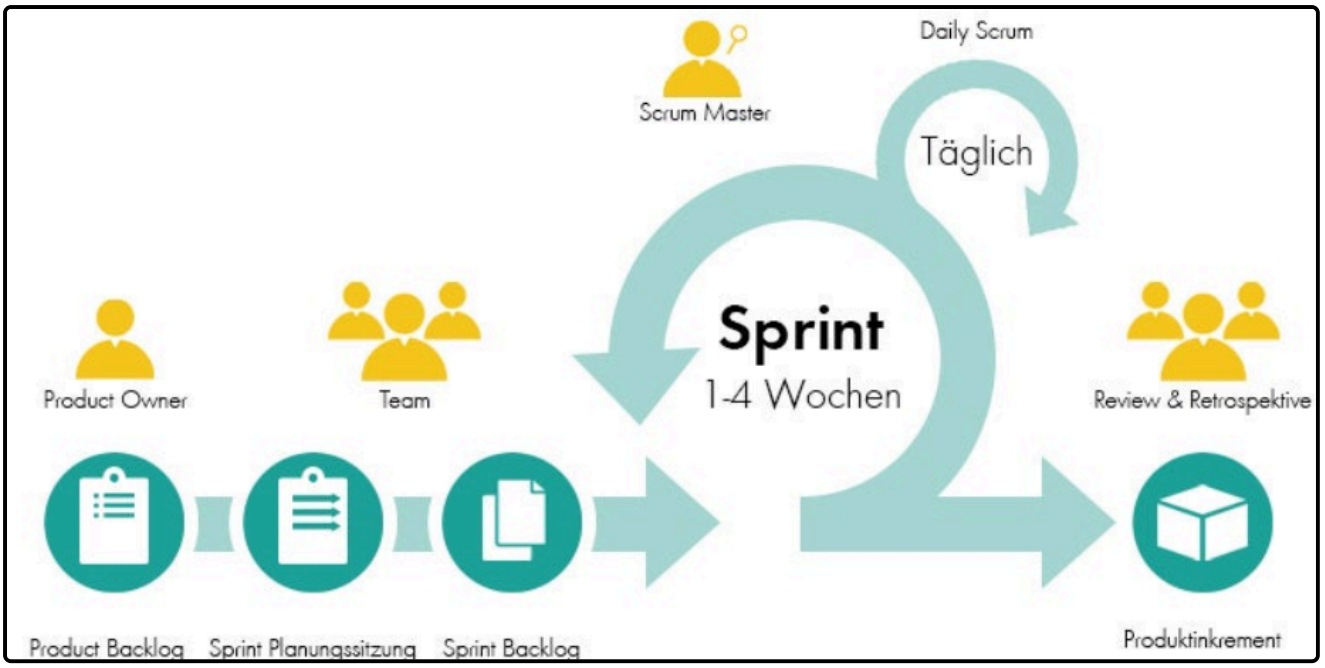
Info:

Konkrete agile Entwicklungsmethoden (**XP**, *Scrum*, etc.) sind kein Bestandteil des Kurses und werden nicht näher konkretisiert.

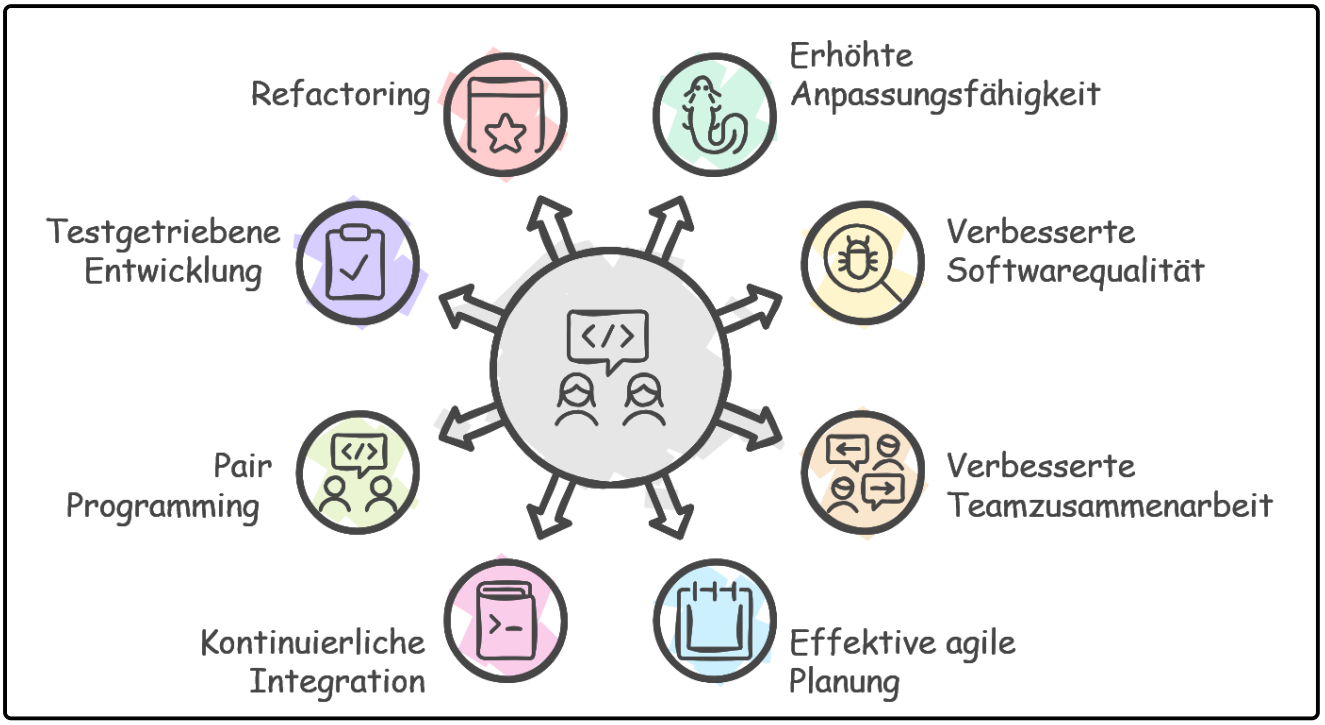
Info:

CI/DI wird ist kein konkreter Bestandteil des Moduls und wird nicht näher konkretisiert.

Grafik: Scrum-Framework



Grafik: XP-Praktiken



CI/CD – Automatisierte Integration und Auslieferung

CI/CD steht für Continuous Integration und Continuous Delivery/Deployment. Diese Verfahren erlauben es, Änderungen am Quellcode fortlaufend zu integrieren, zu testen und automatisiert bereitzustellen.

Continuous Integration (**CI**) ist ein automatisierter Entwicklungsprozess bei dem Codeänderungen kontinuierlich in ein zentrales Repository integriert und automatisch getestet werden.

Continuous Delivery (**CD**) erweitert Continuous Integration und sorgt dafür, dass der Code jederzeit für ein Release bereit ist. Der Code durchläuft automatisierte Integrationstests, Systemtests und Abnahmetests und wird dann in eine staging-ähnliche Umgebung überführt.

Continuous Deployment (**CD**) geht noch einen Schritt weiter als Continuous Delivery. Alle erfolgreich getesteten Änderungen werden automatisch in die Produktionsumgebung ausgerollt ohne dass ein manueller Eingriff nötig ist.

Entwicklungsstratgien, -vorgehensmodelle & -methoden in einem Kontext

Entwicklungsstrategien beschreiben die strategische Grundausrichtung der Organisation, beispielsweise projektbasiert oder produktbasiert. Sie legen fest, in welchem strukturellen Rahmen Entwicklung stattfindet.

Vorgehensmodelle definieren auf operativer Ebene die grundlegende Prozesslogik, etwa planbasiert oder agil. Sie beantworten die Frage, wie Entwicklung gesteuert wird.

Konkrete Vorgehensmethoden sind die praktische Ausprägung dieser Modelle, zum Beispiel das Wasserfallmodell oder das V-Modell als planbasierte Ansätze sowie Scrum oder Kanban als agile Ansätze.

Strategie, Vorgehensmodell und Methode stehen somit in einem hierarchischen Zusammenhang und ergänzen sich gegenseitig.

Tabelle: Stratgien, Vorgehensmodelle & konkrete Methodik in einem Kontext

Ebene	Kategorie	Beispiele
strategisch	Entwicklungsansätze	projektbasiert, produktbasiert
operativ	Vorgehensmodelle	planbasiert, agil
konkret	Vorgehensmethoden	*Wasserfall*-Modell, *V*-Modell, *Scrum*, *Kanban*, **XP**

Die sechs Phasen der Softwareentwicklung und der Softwarelebenszyklus

Softwarelösungen entstehen stets schrittweise über aufeinander aufbauende Phasen. Sechs Phasen sind konzeptionell in allen Vorgehensmodellen wiederzufinden. Diese können je nach konkreter Vorgehensmethode unterschiedlich benannt und umgesetzt werden, die Grundidee bleibt jedoch gleich.

In der projektbasierten Entwicklung erscheinen sie häufig als weitgehend sequenziell durchlaufene Schritte. In produktbasierten Methoden werden sie hingegen in kurzen Iterationen kontinuierlich wiederholt und überlagern sich teilweise. Es wird dann auch von einem Softwarelebenszyklus gesprochen.

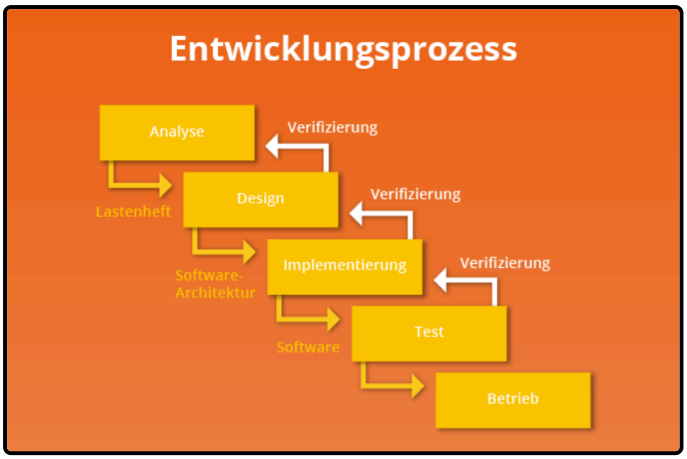
Die sechs Phasen der Softwareentwicklung lauten: Anforderungsphase, Konzeptionsphase, Entwicklungsphase, Qualitätssicherungsphase, Bereitstellungsphase und Wartungsphase.

⚠ **Achtung: Verwirrungsgefahr**

Je nach Quelle wird manchmal von sieben und selten von fünf Phasen gesprochen. Dabei wird eine Phase aufgeteilt in zwei dargestellt oder im Gegenteil, zwei Phasen zu einer zusammengefasst.

Phasen können je nach Quelle auch anders benannt sein, repräsentieren konzeptionell jedoch immer dasselbe.

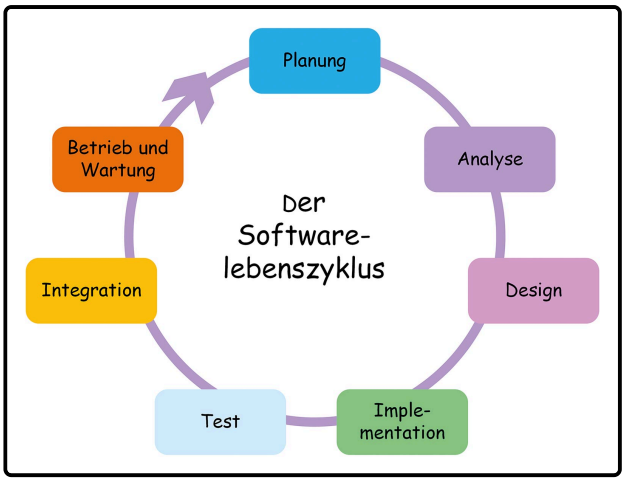
Grafik: Phasen der Softwareentwicklung



⚠ **Achtung: Verwirrungsgefahr**

In dieser Darstellung wurden die Bereitstellungsphase und Wartungsphase zusammengefügt zu "Betrieb".

Grafik: Softwarelebenszyklus



⚠ **Achtung: Verwirrungsgefahr**

In dieser Darstellung wurde die Anforderungsphase in "Planung" und "Analyse" unterteilt.

1. Anforderungsphase

Die Anforderungsphase dient der systematischen Ermittlung und Definition der fachlichen und technischen Anforderungen an ein Softwaresystem. Ziel ist ein klares, gemeinsames Verständnis darüber, was die Software leisten soll und unter welchen Rahmenbedingungen sie entwickelt wird. Sie bildet die Grundlage für alle weiteren Entwicklungsphasen.

Liste: Typische Tätigkeiten

- Anforderungsanalyse und Dokumentation
- Priorisierung der Anforderungen
- Risikoabschätzung und Machbarkeitsanalyse

2. Konzeptionsphase

Basierend auf den Anforderungen wird ein technisches Konzept für die Software entworfen. Dabei werden

4. Qualitätssicherungsphase

Hier wird überprüft, ob die Software den definierten Anforderungen entspricht und frei von Fehlern ist. Falls Fehler gefunden werden, müssen sie in den Code zurückgeführt und behoben werden.

Liste: Typische Tätigkeiten

- Erstellung von Testfällen und Testplänen
- Durchführung von Unit-, Integrations- und Systemtests
- Fehleranalyse und -behebung (Debugging)
- Erstellung von Testdokumentation

5. Bereitstellungsphase

Die Software wird ausgeliefert und installiert Dies schließt die technische sowie

Architektur, Datenstrukturen, Schnittstellen und technologische Grundlagen definiert.

Liste: Typische Tätigkeiten

- Entwurf der Systemarchitektur (**UML**-Diagramme)
- Definition von Datenflüssen und Modulen
- Auswahl geeigneter Technologien und Werkzeuge

> **Info:**
>
> Auf **UML** wird im Detail im Verlauf folgender Veranstaltungen eingegangen.

3. Entwicklungsphase

In dieser Phase erfolgt die tatsächliche Implementierung Der Quellcode wird basierend auf den Konzepten geschrieben und systematisch integriert.

Liste: Typische Tätigkeiten

- Codierung
- Versionierung
- Code-Reviews

organisatorische Vorbereitung der Zielumgebung ein.
Ein reibungsloses Deployment sorgt für einen erfolgreichen Produktstart und minimiert Risiken.

Liste: Typische Tätigkeiten

- Erstellung von Installationspaketen
- Konfiguration der Zielumgebung (Server, Cloud etc.)
- Deployment der Software
- Schulung der Endnutzer

6. Wartungsphase

Nach der Auslieferung folgt der langfristige Betrieb. Dabei müssen sowohl technische Probleme als auch Änderungswünsche berücksichtigt werden.

Liste: Typische Tätigkeiten

- Fehlerbehebung (Bugfixes)
- Einspielen von Sicherheitsupdates
- Performance-Optimierungen
- Weiterentwicklung durch neue Features

> **Achtung:**
>
> Die Beschreibung der Softwarephasen ist allgemein gehalten und vereinfacht.
> Vorallem wird nicht auf die unterschiedlichen Tätigkeitsausprägungen in der planbasierten und produktbasierten Entwicklung eingegangen.

> **Danksagung:**
>
> Liebe Lernende, vielen Dank für die Aufmerksamkeit!
>
> Liebe Veranstalter, vielen Dank für die Bühne!

> **Unterstützung:**
>
> Bei Anmerkung zum Material kontakiert mich gerne unter:
>
> - viktor.korobov@edu.korobov-solutions.de
>
> Unterstützt mich mit Feedback unter:
>
> - Fragebogen